DevOps Lab CA-II: Practical Tasks

Taking a small service through CI/CD, Ansible, Docker + Kubernetes and monitoring

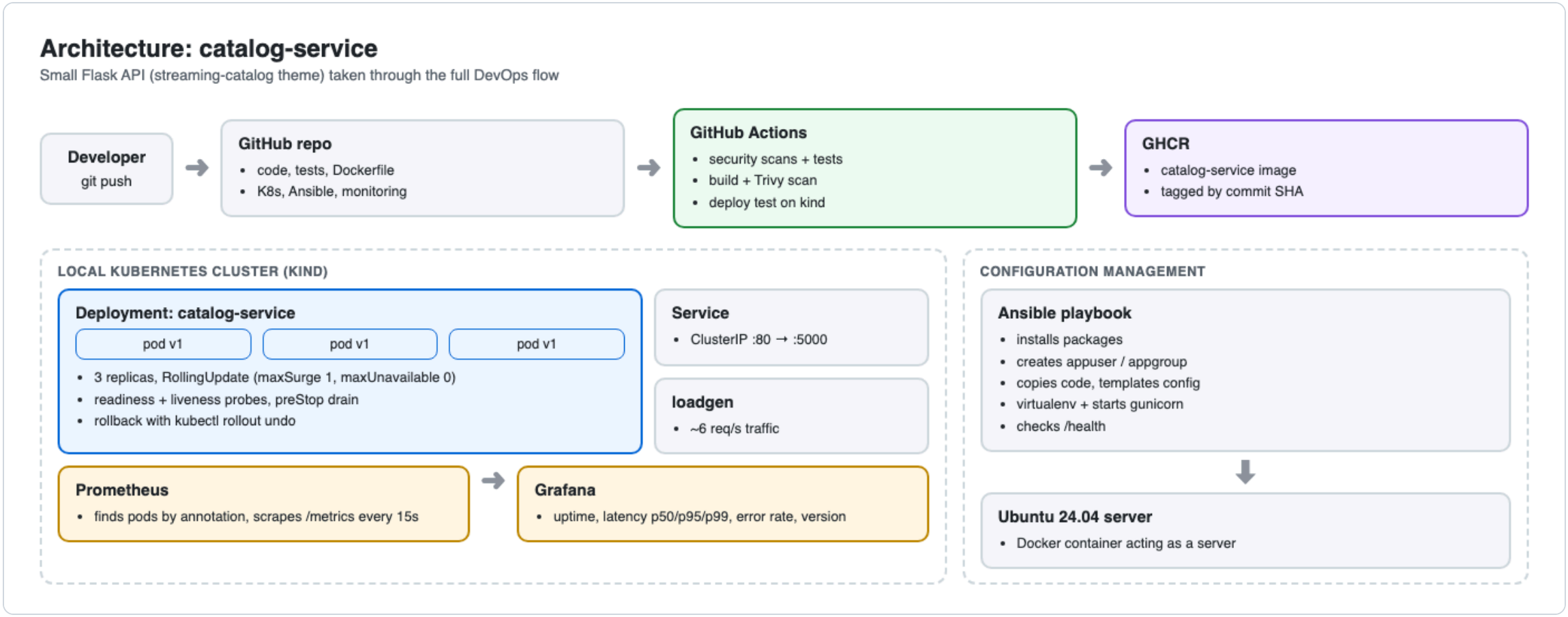
- Service: **catalog-service**, a Flask API with a streaming-catalog theme
- Linked to my written answers: **Q1 Netflix** (resilience) and **Q3 Capital One** (shift-left DevSecOps)
- Repo: github.com/rocko131205/devops-lab-ca2-case-study

4/4	**0 / 160**	**11 → 0**	**100%**
pipeline stages green	failed requests during rolling update	Ansible changes, run 1 → run 2	test coverage (gate: 80%)

Avi S Gupta · B.Tech CSE, Sem VII · DevOps

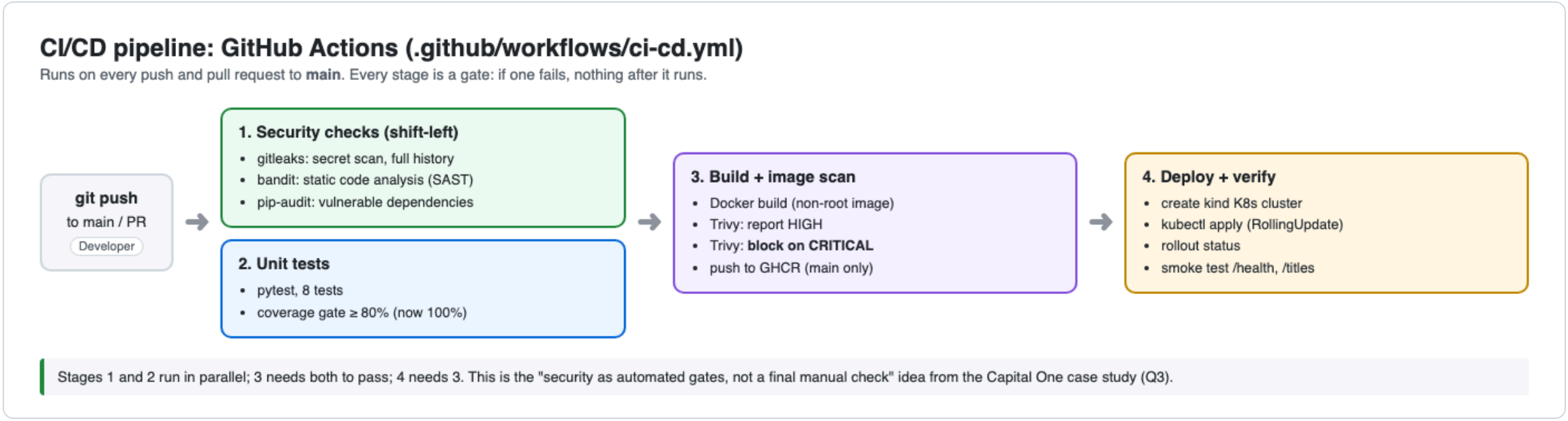
Architecture

Code → pipeline → registry → Kubernetes, with Ansible for server config and Prometheus + Grafana watching it



Pipeline flow (GitHub Actions)

Every stage is an automatic gate. If one fails, nothing after it runs.



Shift-left security (Q3 Capital One)

Secrets (gitleaks), code (bandit), dependencies (pip-audit) and image (Trivy) are all scanned before deploy. Trivy reports HIGH and blocks CRITICAL.

Real deployment check

Each run creates a throwaway Kubernetes cluster, deploys with the same manifests, waits for the rollout, then smoke-tests the API. Full run: about 2m 47s.

Challenges faced

What broke along the way and how I fixed it

3 failed requests in the rolling update

Old pods stopped before traffic moved off them. Added a `preStop` drain (sleep 5). Re-run: 0 of 160 failed.

Secret scan crashed on the first push

The gitleaks Action looks for the parent of the first commit, which a new repo doesn't have. Switched to the gitleaks binary scanning the full history, with the download checksum-verified.

Ansible restart step killed itself

`pkill -f` matched its own shell. Switched to a gunicorn PID file and `pkill -F`.

8 GB laptop

The kube-prometheus-stack chart is heavy. Wrote small Prometheus + Grafana manifests (~400 MB) with resource limits on every pod.

Missing SSL certs in macOS Python

`ansible-galaxy` failed TLS checks. Pointed it at the certifi CA bundle.

Proving zero downtime

Added a watcher pod that hits the Service every 0.25 s during the update and counts which version answered.

Lessons learned

How the practical connects back to the case studies

- **Shift-left is faster:** Trivy flagged 7 fixable HIGH CVEs within a minute of building, not weeks later in a review. The pipeline logs are the audit trail (Q3).
- **Zero downtime needs more than RollingUpdate:** readiness probes, maxUnavailable 0 and a graceful drain.
- **Idempotent config is safe config:** the second Ansible run changed nothing.
- **Resilience has to be visible:** killing a pod and watching it heal, while Grafana shows steady state, is a mini Chaos Monkey (Q1).



Grafana: availability, latency, error rate, and the v1 → v2 → v1 rollout